DevOpsMind AI

Autonomous Software Engineering and CI/CD Intelligence Framework Using AI Agents and Local LLMs

1. Introduction

Modern software development involves multiple activities, including source-code development, code review, security analysis, architecture evaluation, testing, CI/CD execution, deployment, monitoring, troubleshooting, and documentation.

As software projects grow in size and complexity, developers must inspect information from multiple sources such as source code, security reports, architecture dependencies, test results, CI/CD pipelines, build logs, deployment logs, application logs, and project documentation. Manually analysing these sources is time-consuming and can make it difficult to identify the actual root cause of a failure.

DevOpsMind AI is proposed as a privacy-focused, AI-agent-based software engineering and DevOps intelligence framework. The system uses local Large Language Models (LLMs) through Ollama, multi-agent orchestration, Retrieval-Augmented Generation (RAG), GitHub integration, automated testing, static security analysis, CI/CD monitoring, log analysis, and root-cause diagnosis.

The framework consists of specialized AI agents responsible for:
• Code Review
• Security & Vulnerability Analysis
• Architecture & Refactoring
• Test Generation
• CI/CD Monitoring
• Log Analysis
• Documentation & Knowledge Retrieval
• Issue Diagnosis & Correlation
• Report Generation

The agents collectively analyse software-project information and provide developers with identified issues, explanations, possible root causes, recommended actions, and consolidated reports through a Streamlit dashboard.

2. Problem Identification

Software projects generate large amounts of technical information throughout their development and deployment lifecycle.

The major problems identified are:
1. Manual code review requires considerable effort.
2. Hardcoded secrets, API keys, and credentials may be missed.
3. Security vulnerabilities may be detected only at later stages.
4. Architectural smells and circular dependencies are difficult to identify manually.
5. Poor modularity can become difficult to track as projects grow.
6. Test cases are often created manually.
7. Large repositories are difficult to analyse comprehensively.
8. CI/CD failures require manual investigation.
9. Build and deployment logs can contain thousands of lines.
10. Important errors can be buried inside large log files.
11. Developers may need to inspect multiple tools to diagnose one failure.
12. Project documentation can be difficult to locate.
13. Existing monitoring systems often report failures without explaining their causes.
14. Root-cause analysis requires correlation between multiple information sources.
15. Preparing a consolidated technical report requires additional effort.

3. Existing System

In a traditional software-development and DevOps environment, different tools are used for different activities:
• Git/GitHub for source-code management
• Static-analysis tools for code quality
• SAST tools for security analysis
• Dependency-analysis tools for architecture
• Testing frameworks for automated testing
• GitHub Actions or other CI/CD systems for pipelines
• Logging and monitoring tools for operational analysis
• Documentation platforms for project knowledge

When a pipeline fails, developers may need to manually:
1. Identify the failed pipeline.
2. Find the failed stage.
3. Inspect build/test/deployment logs.
4. Identify the error message.
5. Locate the relevant source-code file.
6. Check failed test cases.
7. Examine security or dependency issues.
8. Search project documentation.
9. Determine the probable root cause.
10. Decide the corrective action.

This fragmented workflow increases investigation time and context switching.

4. Limitations of the Existing System

4.1 Manual Code Inspection: Developers must manually inspect source code to identify bugs, code smells, and maintainability issues.

4.2 Security Risks: Hardcoded secrets, exposed credentials, vulnerable patterns, and injection vulnerabilities may be missed.

4.3 Architectural Problems: Circular dependencies, poor modularity, and architectural smells can be difficult to identify manually.

4.4 Manual Test Creation: Developers may need to manually create unit-test cases for important functions.

4.5 Distributed Information: Source code, security reports, tests, pipeline results, logs, and documentation are maintained separately.

4.6 Difficult Log Analysis: Large build and deployment logs make it difficult to identify important errors and events.

4.7 Time-Consuming Diagnosis: Finding the root cause often requires comparing information from multiple sources.

4.8 Limited Monitoring Explanation: Monitoring systems can identify that a pipeline has failed but may not explain why it failed.

4.9 Documentation Discovery: Relevant project information may be difficult to locate manually.

4.10 Manual Report Preparation: Developers may need to consolidate findings from multiple tools into a final report.

5. Proposed System

DevOpsMind AI is an AI-agent-based software engineering and CI/CD intelligence framework designed to unify software analysis, pipeline monitoring, troubleshooting, and reporting.

The system accepts either GitHub repositories, local project folders, or ZIP archives. The project is preprocessed and analysed by specialized agents using locally executed LLMs through Ollama.

The major capabilities include:
- Source-code quality analysis
- SAST security scanning
- Secret detection
- Dependency and architecture analysis
- Automatic test generation
- Test execution and analysis
- CI/CD pipeline monitoring
- Build/deployment log analysis
- Documentation retrieval through RAG
- Multi-agent issue correlation

• Root-cause analysis
• Recommendation generation
• Automated reporting
• Analysis-history storage
• Interactive dashboard visualization

6. Objectives

The main objectives of DevOpsMind AI are:
1. To integrate GitHub repositories and local project uploads.
2. To execute open-source LLMs locally using Ollama.
3. To analyse source-code quality and maintainability.
4. To identify potential logical and structural issues.
5. To detect security vulnerabilities and exposed secrets.
6. To analyse project architecture and dependencies.
7. To identify circular dependencies and modularity problems.
8. To automatically generate PyTest test cases.
9. To execute and analyse generated tests.
10. To monitor CI/CD pipeline execution.
11. To detect failed pipeline stages.
12. To analyse build, test, deployment, and application logs.
13. To extract important errors and events from large logs.
14. To retrieve relevant project documentation using RAG.
15. To correlate findings from multiple agents.
16. To identify probable root causes of software failures.
17. To recommend corrective actions.
18. To generate consolidated software-engineering reports.
19. To store analysis history in SQLite.
20. To present results through an interactive Streamlit dashboard.

7. System Design

The proposed system consists of 10 major components:

Module 1 — Repository Integration:
Responsible for GitHub repository connection, local folder ingestion, ZIP extraction, source-code collection, configuration-file collection, and documentation collection.

Module 2 — AI Code Review Agent:
Analyses source code for code-quality problems, code smells, logical issues, missing error handling, duplication, maintainability problems, and readability issues.

Module 3 — Security & Vulnerability Agent:
Performs SAST-oriented analysis for hardcoded credentials, API keys, tokens, passwords, SQL injection risks, unsafe coding patterns, OWASP-related vulnerabilities, and sensitive information exposure.

Module 4 — Architecture & Refactoring Agent:
Analyses project structure, module dependencies, import relationships, circular dependencies, coupling, modularity, design-pattern concerns, and refactoring opportunities.

Module 5 — Test Generation Agent:
Identifies important functions, analyses signatures, identifies input conditions, generates test scenarios, generates PyTest code, executes tests, and analyses results.

Module 6 — CI/CD Monitoring Agent:
Connects with GitHub Actions and monitors workflow status, build status, test status, deployment status, failed stages, execution duration, and workflow failures.

Module 7 — Log Analysis Agent:
Processes build logs, test logs, deployment logs, and application logs to extract errors, warnings, stack traces, failed commands, important events, and error locations.

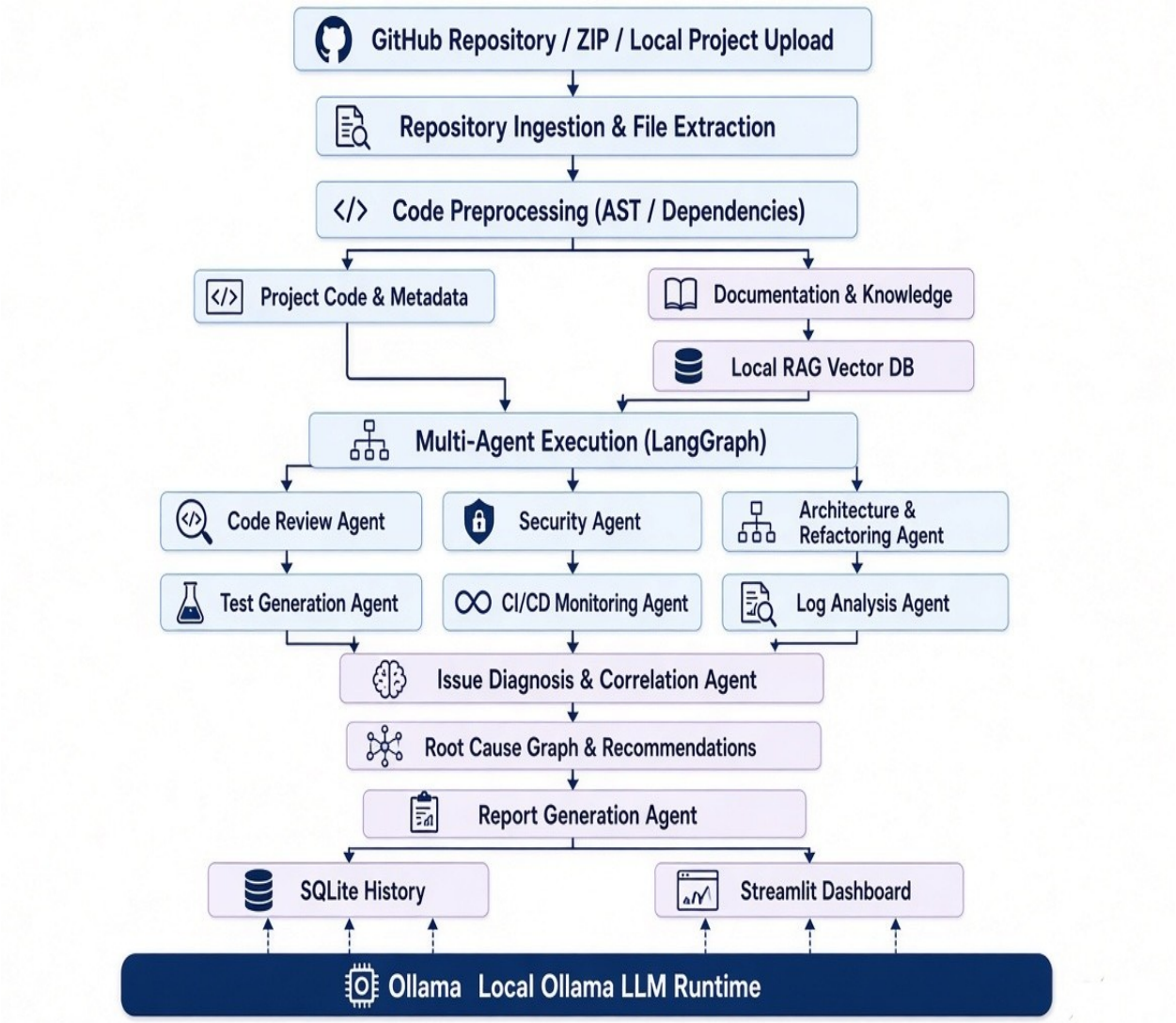
Module 8 — Documentation & Knowledge RAG Agent:
Collects project knowledge from README.md, Markdown files, documentation, and configuration guides using the workflow: Load → Chunk → Embed → Store → Retrieve.

Module 9 — Issue Diagnosis & Correlation Agent:
Central reasoning component that correlates code-review findings, security findings, architecture findings, test results, CI/CD status, log errors, and documentation context to produce root-cause analysis.

Module 10 — Reporting & Dashboard:
Generates final reports, stores results in SQLite, and visualizes findings, pipeline status, test results, security findings, and root-cause graphs in Streamlit.

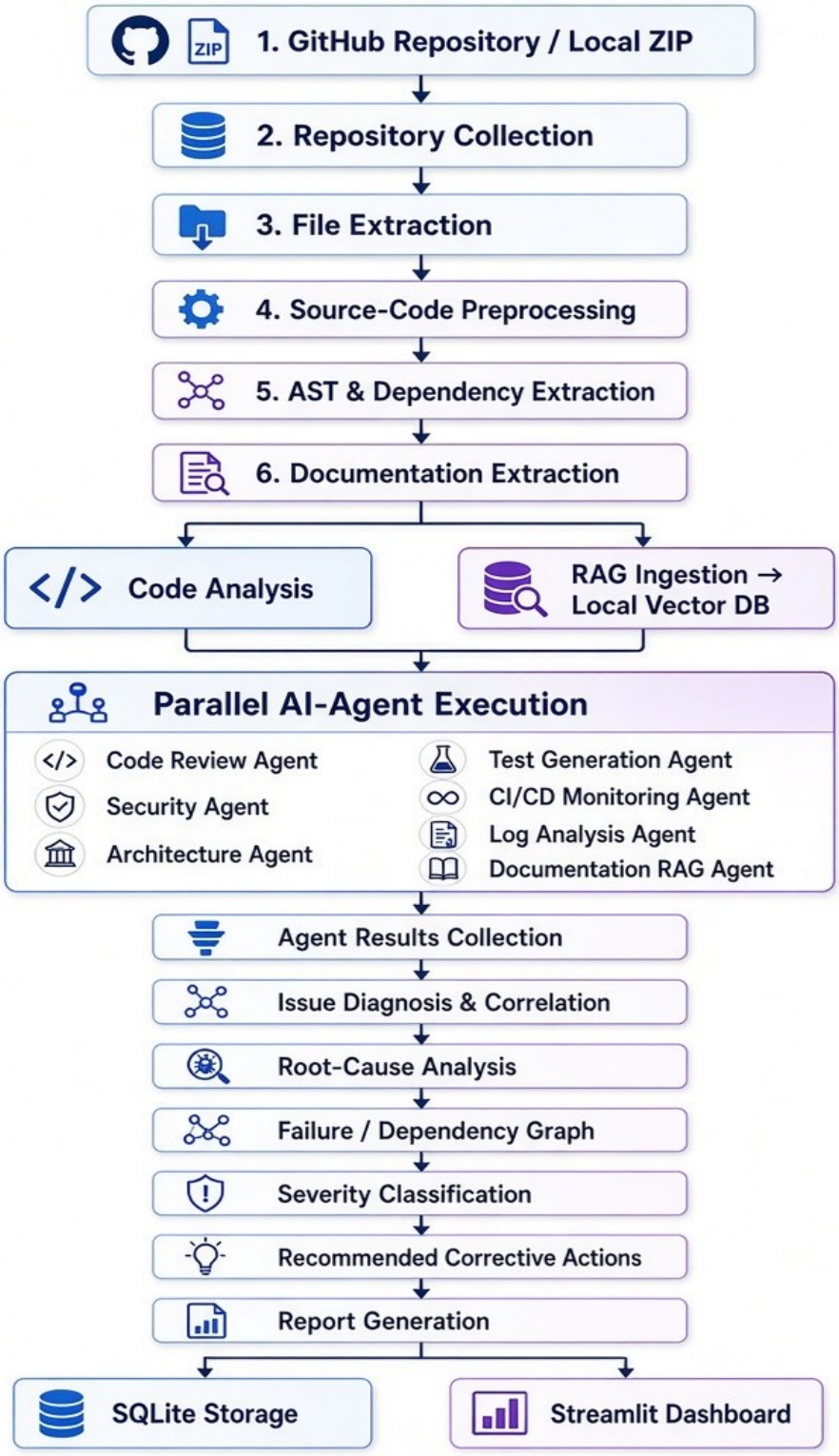
8. System Architecture

The high-level architecture diagram is shown below:



9. System Workflow

The complete sequential system workflow:



10. Algorithm

Algorithm: DevOpsMind AI Multi-Agent Software Analysis

Input: Repository files, Source code, Project documentation, Test results, CI/CD information, Build/Deployment/Application logs.

Output: Code-review findings, Security findings, Architecture findings, Generated tests, Pipeline status, Log analysis, Root-cause graph, Recommendations, Final report.

Step 1 — Repository Collection: Connect to GitHub repository or extract uploaded ZIP/local project.

Step 2 — File Identification: Identify supported files (Python, JavaScript, Java, SQL, Markdown, YAML, Configs).

Step 3 — Source-Code Preprocessing: Extract functions, classes, imports, dependencies, and AST structures.

Step 4 — Knowledge-Base Preparation: Process documentation and store embeddings in local vector DB.

Step 5 — Agent Dispatch: Dispatch preprocessed information to specialized agents via LangGraph.

Step 6 — Code Review: Analyse code quality, maintainability, logical problems, and code smells.

Step 7 — Security Analysis: Detect secrets, credentials, vulnerable patterns, and injection risks.

Step 8 — Architecture Analysis: Analyse dependencies, circular imports, modularity, and refactoring needs.

Step 9 — Test Generation: Generate PyTest scenarios and code for selected functions.

Step 10 — Test Execution: Execute generated tests and collect pass/fail/coverage stats.

Step 11 — CI/CD Collection: Collect GitHub Actions status and identify failed stages.

Step 12 — Log Analysis: Parse build, test, and deployment logs to extract errors and stack traces.

Step 13 — RAG Context Retrieval: Query local documentation using extracted errors.

Step 14 — Diagnostic Correlation: Combine all outputs to map relationships between findings.

Step 15 — Root-Cause Analysis: Determine primary root cause and contributing issues.

Step 16 — Severity Classification: Assign CRITICAL, HIGH, MEDIUM, or LOW severity.

Step 17 — Recommendation Generation: Generate actionable code fixes and patching guidance.

Step 18 — Report Generation: Generate consolidated software-engineering report.

Step 19 — Result Storage: Persist analysis results and metadata in SQLite.

Step 20 — Dashboard Visualization: Render results, status, graphs, and history in Streamlit.

11. Functional Requirements

The system shall:

1. Connect to GitHub repositories.
2. Support local project and ZIP uploads.
3. Extract supported source files.
4. Perform AST-based preprocessing.
5. Execute local LLMs using Ollama.
6. Analyse code quality.
7. Perform SAST security analysis.
8. Detect secrets and exposed credentials.
9. Analyse dependencies and architecture.
10. Detect circular dependencies.
11. Generate PyTest test cases.
12. Execute generated tests.
13. Record test results.
14. Monitor GitHub Actions.
15. Detect failed CI/CD stages.
16. Collect pipeline logs.
17. Analyse build/deployment/application logs.
18. Retrieve documentation through RAG.
19. Correlate multi-agent findings.
20. Generate root-cause analysis.
21. Provide corrective recommendations.
22. Generate consolidated reports.
23. Store analysis history in SQLite.
24. Display results through Streamlit.
25. Visualize dependency and root-cause relationships.

12. Non-Functional Requirements

12.1 Performance: Process project information locally within acceptable time frames based on CPU/GPU capacity.

12.2 Usability: Simple and intuitive Streamlit interface for exploring analysis and root cause results.

12.3 Reliability: Gracefully handle invalid files, broken logs, missing docs, failed tests, and unsupported formats.

12.4 Maintainability: Modular agent structure allowing independent updates to individual agents.

12.5 Security & Privacy: Keep code, logs, docs, and analysis completely local without external API exposure.

12.6 Scalability: Support seamless addition of future agents (Performance, License, Cost Optimization agents).

13. Technology Stack

Area	Technology
Programming	Python 3.11+
Source Control	Git / GitHub
CI/CD	GitHub Actions
LLM Runtime	Ollama
Local LLMs	Code Llama / Llama / Qwen Coder
Agent Framework	LangGraph
LLM Framework	LangChain
RAG	LangChain + Local Embeddings
Embeddings	Hugging Face
Vector Database	FAISS / ChromaDB
Code Analysis	Python AST / Static Analyzers
Testing	PyTest
Database	SQLite
Data Processing	Pandas
Dashboard	Streamlit
Graph Visualization	PyVis / Graphviz / Plotly

14. Feasibility Analysis

14.1 Technical Feasibility: The system utilizes open-source components (Python, Ollama, LangChain, FAISS, SQLite, Streamlit) eliminating cloud API reliance.

14.2 Economic Feasibility: Minimal operational cost due to local model execution and open-source tooling.

14.3 Operational Feasibility: Streamlines developer workflows by merging analysis, logging, and monitoring into one interface.

14.4 Time Feasibility: Planned across 6 structured phases over a 12-week timeframe.

15. Performance and Evaluation

15.1 Code Review Evaluation: Evaluated on benchmark codebases for True/False Positive Rates and recommendation quality.

15.2 Security Evaluation: Measured against precision, recall, and F1-score on known vulnerability benchmarks.

15.3 Architecture Evaluation: Evaluated on circular dependency accuracy and modularity suggestions.

15.4 Test Generation Evaluation: Evaluated on test execution success rate, code coverage, and edge cases.

15.5 CI/CD & Log Analysis Evaluation: Evaluated on log error extraction and failed stage identification.

15.6 Root-Cause Diagnosis Evaluation: Measured on diagnostic precision and cause identification accuracy.

15.7 Local LLM Performance: Evaluated on response latency, token speed, RAM/VRAM footprint, and accuracy.

16. Scope

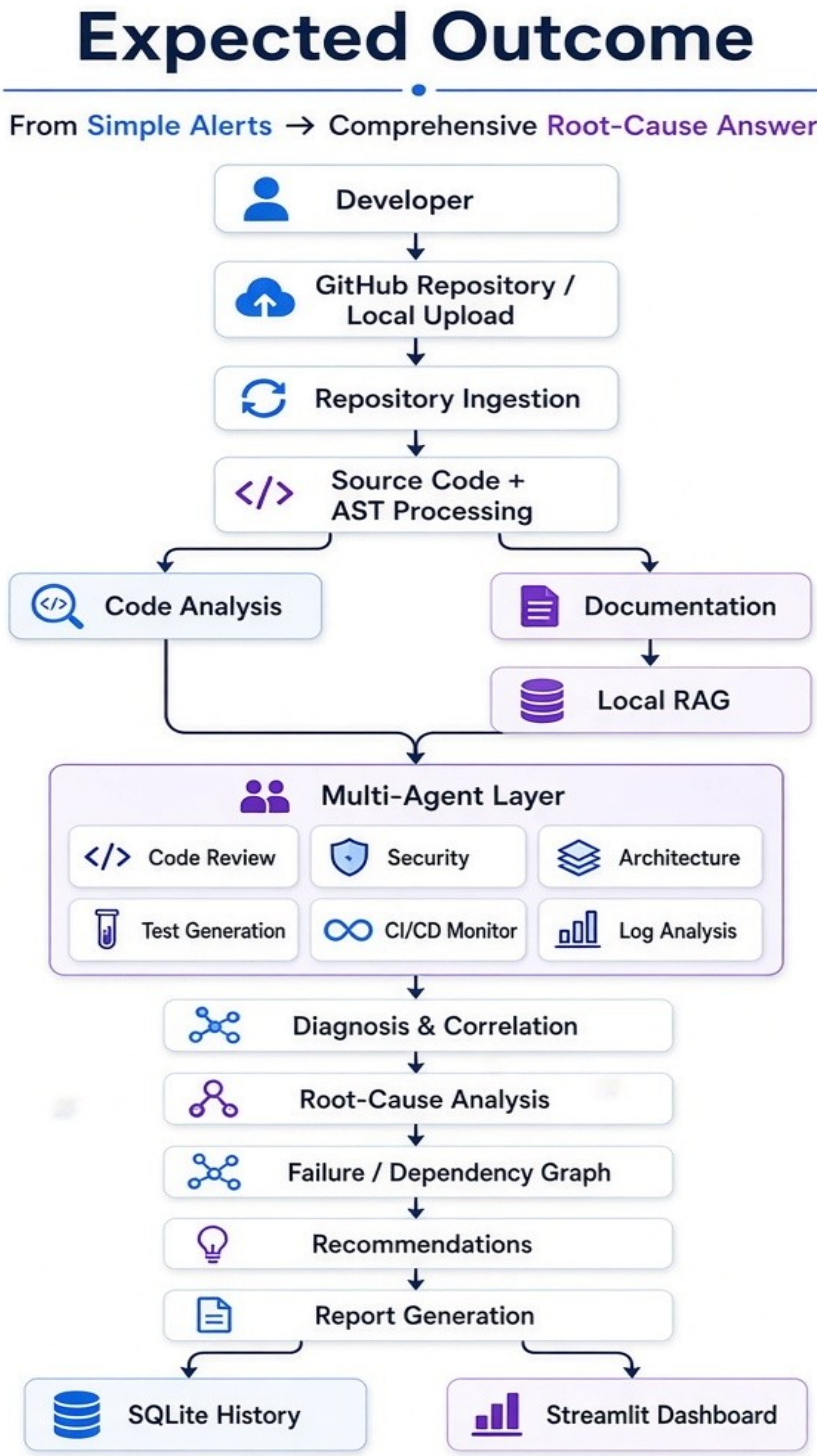
The project covers: GitHub ingestion, local ZIP ingestion, source code preprocessing, AST extraction, local LLM execution, code review, SAST security analysis, secret scanning, architecture analysis, test generation/execution, CI/CD monitoring, log parsing, documentation RAG, root-cause correlation, recommendation generation, report generation, SQLite history, and Streamlit visualization.

17. Limitations

1. Local LLM execution performance depends on available hardware resources (CPU/RAM/GPU).
2. Large repositories require intelligent file filtering and chunking.
3. LLM-based analysis can produce false positives or negatives.
4. SAST agent does not replace formal professional security audits.
5. Test quality depends on context availability and code complexity.
6. RAG effectiveness depends on project documentation depth.
7. Extremely large log files may exceed model context limits.
8. CI/CD integration requires GitHub Actions access permissions.
9. Results may vary depending on the chosen local model size.
10. Root-cause outputs are probabilistic recommendations based on evidence.

18. Expected Outcome

DevOpsMind AI enables developers to move from simple failure alerts ('The pipeline failed') to comprehensive root-cause answers ('The pipeline failed because X; related evidence is Y; probable cause is Z; recommended action is A').



19. Revised Agent Architecture

Specialized AI Agents (9):

1. Code Review Agent
2. Security & Vulnerability Agent
3. Architecture & Refactoring Agent
4. Test Generation Agent
5. CI/CD Monitoring Agent
6. Log Analysis Agent
7. Documentation & Knowledge RAG Agent
8. Issue Diagnosis & Correlation Agent
9. Report Generation Agent